

Two Sum:

Problem: Given an array of integers, return indices of the two numbers such that they add up to a specific target.

Approach: Use a hash map to store complements (target - current element) for $O(n)$ time complexity.

Variants: Two Sum II (sorted array), Three Sum, Four Sum.

Maximum Subarray Sum (Kadane's Algorithm):

Problem: Find the contiguous subarray with the largest sum.

Approach: Use Kadane's algorithm for $O(n)$ time complexity.

Variants: Maximum product subarray, circular subarray sum.

Merge Intervals:

Problem: Given a collection of intervals, merge all overlapping intervals.

Approach: Sort intervals by start time and merge overlapping ones.

Trapping Rain Water:

Problem: Given an elevation map, compute how much water it can trap after raining.

Approach: Use two-pointer technique or dynamic programming.

Find Missing Number:

Problem: Given an array containing n distinct numbers taken from 0 to n , find the missing number.

Approach: Use XOR or the sum of natural numbers formula.

Rotate Array:

Problem: Rotate an array to the right by k steps.

Approach: Use reversal algorithm or cyclic replacements.

Search in Rotated Sorted Array:

Problem: Given a rotated sorted array, find the index of a target element.

Approach: Use modified binary search.

Product of Array Except Self:

Problem: Given an array, return an array where each element is the product of all elements except itself.

Approach: Use prefix and suffix products.

Find All Duplicates in an Array:

Problem: Given an array of integers where $1 \leq a[i] \leq n$, find all duplicates.

Approach: Use index manipulation or hash maps.

Longest Consecutive Sequence:

Problem: Given an unsorted array, find the length of the longest consecutive sequence.

Approach: Use a hash set for $O(n)$ time complexity.

Next Permutation:

Problem: Given an array, rearrange it into the lexicographically next greater permutation.

Approach: Use two-pointer technique and reverse operations.

Sliding Window Maximum:

Problem: Given an array and a window size, find the maximum in each sliding window.

Approach: Use a deque for $O(n)$ time complexity.

Find the Duplicate Number:

Problem: Given an array containing $n+1$ integers where each integer is between 1 and n , find the duplicate.

Approach: Use Floyd's Tortoise and Hare (cycle detection).

Count Inversions:

Problem: Count the number of inversions in an array (i.e., pairs where $i < j$ and $arr[i] > arr[j]$).

Approach: Use modified merge sort.

Subarray Sum Equals K:

Problem: Given an array of integers, find the total number of subarrays whose sum equals k .

Approach: Use prefix sum and hash maps.